

Mini Internet Protocol Stack Simulator

Code Structure and Design Documentation

CITS3002 Project

Group members

- David Do (24224251)
- Soham Parikh (24472235)

1. Submitted Source Code Organization

```
CITS3002_Project/  
  main.py      - required executable entry point  
  config.py    - fixed constants, IP/MAC addresses, routing tables, and MAC tables  
  protocol.py  - Layer 2, Layer 3, and Layer 4 data structures  
  devices.py   - Device, Host, and Router behaviour  
  README.md    - run instructions and project explanation
```

File	Purpose in the implementation
main.py	Starts the simulator. It validates the command-line input, builds the Host A - Router R1 - Host B topology, and starts transmission from Host A to Host B.
config.py	Stores fixed project data such as protocol constants, addresses, ports, routing tables, MAC lookup tables, and interface names.
protocol.py	Defines the objects that represent the Layer 4 segment, Layer 3 packet, and Layer 2 frame. These objects store the header fields and payloads used during encapsulation.
devices.py	Implements the behaviour of Device, Host, and Router. This file contains most of the send, receive, routing, forwarding, checksum, ACK, and logging logic.
README.md	Explains how to run the simulator, what each file does, the topology, expected behaviour, assumptions, and suggested test commands.

2. main.py Design

The file main.py is intentionally kept small because its job is only to set up and run the simulation. The project requirement says main.py is the file called for execution, so the program starts here when it runs a command such as python main.py 100.

Component in main.py	What it does in our code
print_usage()	Prints the required command format if the user gives invalid input.
parse_message_size()	Reads sys.argv, checks that exactly one argument is provided, converts it to an integer, and rejects non-numeric, zero, or negative values.
build_network()	Creates Host A, Host B, and Router R1. It adds Router R1 Interface 1 and Interface 2, then connects Host A to Interface 1 and Host B to Interface 2.
main()	Calls parse_message_size(), builds the topology, prints a short simulation header, and starts the transmission by calling host_a.send_application_data(...).
if __name__ == "__main__"	Ensures the program runs only when main.py is executed directly.

The important design choice in `main.py` is that it does not implement protocol behaviour itself. It imports `Host` and `Router` from `devices.py`, and it imports all fixed values from `config.py`. This keeps setup separate from the Layer 2, Layer 3, and Layer 4 logic.

3. config.py Design

The file `config.py` centralizes all fixed values required by the simulator. This makes the topology and protocol settings easy to inspect and prevents hard-coded addresses from being scattered across the project.

Group of values in config.py	Examples	Purpose
Protocol constants	ETHERNET_TYPE_IPV4 = 0x0800; IP_PROTOCOL_UDP = 17; DEFAULT_TTL = 100; L4_TYPE_DATA = 0; L4_TYPE_ACK = 1	Defines fixed protocol values used by the frame, packet, and segment classes.
Size constants	MAX_SEGMENT_DATA_SIZE = 500; TRANSPORT_HEADER_SIZE = 10; NETWORK_HEADER_SIZE = 12; DATA_LINK_HEADER_SIZE = 14	Used for segmentation and length calculations.
IP addresses	HOST_A_IP = 10.0.1.10; ROUTER_R1_INTERFACE_1_IP = 10.0.1.1; ROUTER_R1_INTERFACE_2_IP = 10.0.2.1; HOST_B_IP = 10.0.2.20	Defines the Layer 3 addressing scheme from the project topology.
MAC addresses	HOST_A_MAC; ROUTER_R1_INTERFACE_1_MAC; ROUTER_R1_INTERFACE_2_MAC; HOST_B_MAC	Defines the Layer 2 hardware addresses for local frame delivery.
Port numbers	HOST_A_PORT = 5000; HOST_B_PORT = 80	Defines the transport-layer source and destination ports.
MAC lookup tables	HOST_A_MAC_TABLE, HOST_B_MAC_TABLE, ROUTER_R1_MAC_TABLE	Map next-hop IP addresses to destination MAC addresses. Layer 2 uses these tables before creating a frame.
Routing tables	HOST_A_ROUTING_TABLE, HOST_B_ROUTING_TABLE, ROUTER_R1_ROUTING_TABLE	Map destination networks to next-hop IP addresses and outgoing interfaces. Layer 3 uses these tables for forwarding decisions.

The corrected TTL update is handled in one place by setting `DEFAULT_TTL = 100`. Because `devices.py` uses `DEFAULT_TTL` when creating every `Layer3Packet`, changing this one constant updates the output logs consistently.

4. protocol.py Design

The file `protocol.py` contains the classes that model the protocol units passed between layers. The code does not send raw bytes over a real network. Instead, it uses Python objects to represent each header and payload.

4.1 Layer4Segment

`Layer4Segment` represents the UDP-like transport segment with ACK support. It is the lowest object created by the application-facing send logic before the data is placed inside a `Layer3Packet`.

Field or method	Purpose in the code
src_port and dst_port	Represent the sending and receiving transport-layer ports.
segment_type	Stores whether the segment is DATA or ACK using L4_TYPE_DATA or L4_TYPE_ACK.
seq_num	Stores the alternating-bit sequence number, either 0 or 1.
data	Stores the application payload as bytes. ACK segments use empty data.
length	Calculated as TRANSPORT_HEADER_SIZE plus the number of data bytes.
checksum	Computed when the segment is created unless a checksum is explicitly supplied.
compute_checksum()	Builds a stable 16-bit checksum from the segment header fields and data.
verify_checksum()	Recomputes the checksum and compares it with the stored checksum.
is_data() and is_ack()	Allow receive logic to branch clearly between DATA and ACK handling.
get_data_size()	Returns the number of application data bytes for logging delivery size.

4.2 Layer3Packet

Layer3Packet represents the IP-like network packet. It wraps a Layer4Segment and stores the source and destination IP addresses used for routing.

Field or method	Purpose in the code
src_ip and dst_ip	Represent the end-to-end IP addresses. For DATA, these are Host A and Host B. For ACK, these are Host B and Host A.
ttl	Starts at DEFAULT_TTL, which is 100. Router R1 decrements it when forwarding.
protocol	Stores IP_PROTOCOL_UDP = 17 to represent a UDP-like Layer 4 payload.
payload	Stores the Layer4Segment being carried by the packet.
total_length	Calculated as NETWORK_HEADER_SIZE plus the length of the Layer4Segment.
decrement_ttl()	Reduces TTL by one and returns both old and new values so Router R1 can log the exact change.
is_expired()	Returns True if the packet should be dropped because TTL has reached zero.

4.3 Layer2Frame

Layer2Frame represents the Ethernet-like data-link frame. It wraps a Layer3Packet and carries the local source and destination MAC addresses for the next hop.

Field or method	Purpose in the code
src_mac and dst_mac	Represent local-link MAC addresses for the current hop. These change when Router R1 forwards the packet.
frame_type	Stores ETHERNET_TYPE_IPV4 = 0x0800 to show the payload is an IPv4-like packet.
payload	Stores the Layer3Packet being carried by the frame.
length	Calculated as DATA_LINK_HEADER_SIZE plus the Layer3Packet total length.
__repr__()	Provides a readable representation for debugging.

5. devices.py Design

The file devices.py contains the behaviour of the simulated devices. It is the main implementation file because it controls how frames, packets, and segments move between Host A, Router R1, and Host B.

5.1 Device Base Class

Device is a shared base class used by both Host and Router. Its purpose is to avoid repeating common code.

Component in Device	Purpose in the code
---------------------	---------------------

name	Display name used in logs.
routing_table	Layer 3 route entries for this device.
mac_table	Layer 2 next-hop IP to MAC mappings.
learned_mac_table	MAC addresses learned from incoming frames.
log(layer, message)	Prints Device: Layer: Operation.
lookup_route(destination_ip)	Returns next-hop IP and outgoing interface.
lookup_mac(next_hop_ip)	Returns MAC address for the next-hop IP.

5.2 Host Class

Host represents both Host A and Host B. The same class is used for both because both hosts need to send and receive segments, packets, and frames. Host A starts as the sender, while Host B receives DATA and sends ACKs back.

Host component	Purpose in the code
ip_address and mac_address	Store the host Layer 3 and Layer 2 addresses.
connected_router and connected_router_interface	Store the logical connection from the host to Router R1.
next_send_seq	Stores the next DATA sequence number to send. It alternates between 0 and 1 after correct ACKs.
expected_receive_seq	Stores the DATA sequence number expected when this host is receiving data.
last_ack_received	Stores the ACK sequence number received by the sender.
last_ack_sent	Stores the last ACK sequence number sent, used if a duplicate or corrupted segment is handled.

Host method	What it does in the implementation
connect_to_router()	Connects the host to Router R1 and records the router interface used for sending frames.
send_application_data()	Receives the input message size, creates dummy byte data, splits it into chunks of at most 500 bytes, creates DATA segments, and sends each segment sequentially.
_send_data_segment_until_ack()	Implements stop-and-wait behaviour. It sends the current DATA segment and waits until the correct ACK sequence number is received.
send_segment_to_network()	Encapsulates a Layer4Segment inside a Layer3Packet, performs routing lookup, and passes the packet down to Layer 2.
send_packet_to_data_link()	Looks up the destination MAC address, creates a Layer2Frame, logs frame creation, and passes the frame to Router R1.
receive_frame()	Accepts frames addressed to the host MAC, learns the source MAC, and delivers the packet to Layer 3.
receive_packet_from_data_link()	Checks whether the destination IP matches the host IP. If it does, the segment is delivered to Layer 4.
receive_segment_from_network()	Verifies checksum, handles ACKs, delivers valid DATA to the application layer, detects duplicates, and sends ACKs.
_send_ack()	Creates an ACK segment with the same sequence number as the accepted DATA segment and sends it back to the source IP.

5.3 Router Class

Router represents Router R1. It has two interfaces and forwards packets between the two subnets. It does not deliver application data; it only forwards packets.

Router component or method	Purpose in the code
interfaces	Dictionary storing Interface 1 and Interface 2, including IP address, MAC address, and connected host.
add_interface()	Registers a router interface with its IP and MAC address.
connect_interface()	Links a router interface to Host A or Host B in the logical topology.
receive_frame()	Receives a Layer2Frame on a specific interface, validates the destination MAC, learns the source MAC, and passes the packet to Layer 3.
receive_packet_from_data_link()	Logs the packet, decrements TTL, drops expired packets, performs routing lookup, and passes the packet back down to Layer 2 for forwarding.
send_packet_to_data_link()	Creates a new Layer2Frame using the outgoing router interface MAC as the source MAC and the next-hop MAC as the destination MAC. Then it sends the frame to the connected host.

6. How the Files Work Together During One Run

The following sequence describes the actual code path when the command `python main.py 10` is run.

1. `main.py` calls `parse_message_size()` to read the value 10 from the command line.
2. `main.py` calls `build_network()`, which creates Host A, Host B, and Router R1 using values imported from `config.py`.
3. `build_network()` calls `Router.add_interface()` twice to add Interface 1 and Interface 2.
4. `build_network()` calls `Host.connect_to_router()` and `Router.connect_interface()` to connect both logical links.
5. `main.py` calls `host_a.send_application_data(data_size=10, destination_ip=HOST_B_IP)`.
6. `Host.send_application_data()` creates a `Layer4Segment` with DATA, `seq=0`, and 10 bytes of data.
7. `Layer4Segment.compute_checksum()` calculates the transport checksum.
8. `Host.send_segment_to_network()` wraps the segment in a `Layer3Packet` with `TTL=100`.
9. `Device.lookup_route()` selects Router R1 Interface 1 as Host A's next hop.
10. `Host.send_packet_to_data_link()` uses `Device.lookup_mac()` to find `BB:BB:BB:BB:BB:BB` and creates a `Layer2Frame`.
11. `Router.receive_frame()` receives the frame on Interface 1, validates the destination MAC, and learns Host A's MAC address.
12. `Router.receive_packet_from_data_link()` decrements TTL from 100 to 99 and routes the packet toward Host B through Interface 2.
13. `Router.send_packet_to_data_link()` creates a new frame with source MAC `CC:CC:CC:CC:CC:CC` and destination MAC `DD:DD:DD:DD:DD:DD`.
14. Host B `receive_frame()` accepts the frame, learns the router MAC, and passes the packet to Layer 3.
15. Host B `receive_packet_from_data_link()` sees that the destination IP is 10.0.2.20, so it delivers the segment to Layer 4.
16. Host B `receive_segment_from_network()` verifies the checksum, delivers 10 bytes to the application layer, and calls `_send_ack()`.
17. The ACK follows the reverse path through Host B, Router R1, and Host A.

18. Host A `receive_segment_from_network()` receives ACK `seq=0`, allowing the sender to complete or move to the next segment.

7. Segmentation and Reliability Design in the Code

The transport-layer reliability is implemented mainly in `Host.send_application_data()`, `Host._send_data_segment_until_ack()`, `Host.receive_segment_from_network()`, and `Host._send_ack()`.

Input command	Segment sizes created by <code>Host.send_application_data()</code>	Sequence numbers used
<code>python main.py 10</code>	10	0
<code>python main.py 500</code>	500	0
<code>python main.py 501</code>	500, 1	0, 1
<code>python main.py 1200</code>	500, 500, 200	0, 1, 0

The code uses stop-and-wait behaviour. After each DATA segment is sent, Host A waits until `last_ack_received` matches the current segment sequence number. Only then does Host A alternate `next_send_seq` and send the next segment. This directly implements the required rdt2.2 alternating-bit behaviour.

8. Mapping of Project Requirements to Code

Requirement area	Where it is implemented in the code
Layer 2 frame creation and MAC addressing	Layer2Frame in protocol.py; Host.send_packet_to_data_link(); Router.send_packet_to_data_link(); MAC tables in config.py.
MAC learning	Host.receive_frame() and Router.receive_frame() update learned_mac_table.
Layer 3 packet creation	Layer3Packet in protocol.py; Host.send_segment_to_network().
Routing table lookup	Device.lookup_route() in devices.py uses the routing tables from config.py.
TTL decrement and expiry	Layer3Packet.decrement_ttl(), Layer3Packet.is_expired(), and Router.receive_packet_from_data_link().
Layer 4 DATA and ACK segments	Layer4Segment in protocol.py; Host.send_application_data(); Host._send_ack().
Segmentation into maximum 500-byte payloads	Host.send_application_data() uses MAX_SEGMENT_DATA_SIZE from config.py.
Checksum computation and verification	Layer4Segment.compute_checksum() and Layer4Segment.verify_checksum().
rdt2.2 alternating sequence numbers	Host.next_send_seq, Host.expected_receive_seq, Host._send_data_segment_until_ack(), and Host.receive_segment_from_network().
Logging requirement	Device.log() provides a shared logging format used throughout Host and Router methods.
Program execution through main.py	main.py contains parse_message_size(), build_network(), and main().

9. Code Documentation Standard

The code has been documented to make the structure and design clear. The documentation is not only theory: it explains the purpose of files, classes, and methods in the actual implementation.

- Each Python file begins with a module docstring explaining the purpose of that file.
- Each main class has a class docstring explaining what it represents in the simulator.
- Each important method has a function docstring explaining what it does.
- Inline comments are used for non-obvious implementation decisions, such as MAC lookup, directly connected routes, TTL handling, checksum computation, and alternating-bit sequence changes.
- README.md explains how to run the code, what each file does, and which test commands should be used.

10. Testing and Expected Results

The following commands should be run from the project directory to check the main behaviours.

```
python main.py 10
python main.py 500
python main.py 501
python main.py 1200
```

Test	What it checks
python main.py 10	Matches the project sample size and should produce one DATA segment and one ACK.
python main.py 500	Checks the maximum single segment size.
python main.py 501	Checks the segmentation boundary by producing a 500-byte segment and a 1-byte segment.
python main.py 1200	Checks multiple sequential segments: 500, 500, and 200 bytes with sequence numbers 0, 1, 0.

For a successful run, the logs should show Host A sending DATA, Router R1 forwarding it, Host B delivering the DATA to the application layer, Host B sending an ACK, Router R1 forwarding the ACK, and Host A receiving the correct ACK.

11. Conclusion

The implementation is organized so the purpose of each file and component is clear. main.py handles execution, config.py stores fixed settings, protocol.py defines the protocol data objects, devices.py implements Host and Router behaviour, and README.md explains how to run and test the program. The main components directly support the required Layer 2 frame delivery, Layer 3 routing and TTL handling, and Layer 4 segmentation, checksum, ACK, and rdt2.2 reliability features.